

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Distribuidos

Mexi Banco: dos bases más y una caché
Persistencia políglota con Redis y MongoDB

Bitácora del laboratorio S11

Alumno: Christian Franco
Cuenta: 117003136
Materia: Sistemas Distribuidos
Grupo: Grupo 2
Fecha: 7 Noviembre de 2026

Parte 0: Ver el punto de partida

Antes de agregar Redis y MongoDB medimos cómo se comportaba el sistema base, para tener contra qué comparar.

0.1 Diez consultas seguidas al saldo de Ana

Hicimos 10 peticiones `GET` consecutivas al gateway para consultar el saldo de Ana (`002180000000000001`):

```
for i in $(seq 1 10); do curl -s localhost:8080/api/cuentas/002180000000000001;
    echo; done
```

Las diez respuestas fueron iguales en el saldo (`1000.00`) y en el campo `leidaEn`, que cambia en cada línea de la salida real porque cada lectura va a la base de datos:

```
{"clabe":"002180000000000001","titular":"Ana","saldo":1000.00,"leidaEn
    ":"2026-11-07T03:30:02.318498997Z"}
(...se repite 10 veces)
```

0.2 Cuántas lecturas llegaron a PostgreSQL

Revisamos los registros de las tres instancias del servicio `cuenta` para contar cuántas consultas llegaron realmente a la base relacional:

```
docker compose logs cuenta | grep -c "leida de Postgres"
```

El resultado fue `10`: cada consulta pegó directo en PostgreSQL.

Con este punto de partida queda claro el problema: sin caché, cada vez que alguien abre la app para ver su saldo se ejecuta un `SELECT` en PostgreSQL. Con 10,000 clientes consultando al mismo tiempo, la base recibiría 10,000 lecturas que compiten por conexiones y bloqueos con las transacciones que sí son críticas (cargos, abonos y transferencias).

0.3 Estado inicial de Redis y MongoDB

Por último verificamos que Redis no tuviera claves en uso y que la colección de avisos de MongoDB estuviera vacía:

```
docker compose exec redis redis-cli INFO stats | grep keyspace
docker compose exec mongo mongosh notificacion --eval 'db.avisos.countDocuments()
    '
```

La salida fue `keyspace_hits:0`, `keyspace_misses:0` y `0` documentos, es decir, ambos partían desde cero.

Parte 1: La caché del saldo en Redis

1.1 Diseño e implementación

Implementamos el patrón Cache-Aside en el microservicio `cuenta` con tres cambios:

1. **`ConfiguracionCache.java`**: se anotó con `@EnableCaching` y se configuró la caché `cuentas` con un TTL de 30 segundos (`mexibanco.cache.ttl:30s`), sin guardar valores nulos (`disableCachingNullValues()`) y serializando en JSON legible con `Jackson2JsonRedisSerializer`. También se activó `transactionAware()`, para que las invalidaciones (`DEL`) ocurran **solo después de que PostgreSQL haga commit**.
2. **`ConsultaDeSaldo.java`**: el método `consultar(String clabe)` se anotó con `@Cacheable(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")`. Si hay *hit*, responde Redis en unos 3 ms sin tocar PostgreSQL. Si hay *miss*, se ejecuta el método, se consulta PostgreSQL, se escribe el log `[consulta] <clabe>leida de Postgres` y Spring guarda el resultado en Redis con su TTL.
3. **`CuentaService.java`**: los métodos `cargar(...)` y `abonar(...)` se anotaron con `@CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")`. Cada vez que cambia un saldo se borra su clave en Redis, y la siguiente lectura recarga el valor fresco desde PostgreSQL.

Cabe aclarar que el método `CuentaService.consultar`, que decide si un cargo procede, vive en una clase separada y **nunca** usa la caché: siempre va directo a PostgreSQL. Así evitamos autorizar un movimiento con un saldo viejo.

1.2 Levantar el entorno y comprobar Redis

Después de aplicar los cambios reconstruimos las imágenes y verificamos que Redis respondiera antes de hacer cualquier medición:

```
docker compose up -d --build
docker compose ps
docker compose exec redis redis-cli PING
```

El comando `PING` responde `PONG`, con lo que confirmamos que la caché estaba disponible para las instancias de `cuenta`.

1.3 Una caché compartida por todas las instancias

Hicimos seis consultas seguidas al saldo de Ana, mostrando solo la cabecera `X-Instancia` y el campo `leidaEn`:

```
for i in 1 2 3 4 5 6; do curl -s -D - localhost:8080/api/cuentas
   /002180000000000001 | grep -iE 'x-instancia|leidaEn'; done
```

La cabecera `X-Instancia` rotó entre las tres réplicas (`f99ec8e404f5`, `31597067bb62` y `f31fa76b3e9b`), pero `leidaEn` fue idéntico en las seis respuestas. Esto confirma que las tres instancias leen la misma entrada de Redis: la caché es del sistema, no de cada proceso.

Después inspeccionamos directamente la clave en Redis:

```
docker compose exec redis redis-cli GET cuentas::002180000000000001
docker compose exec redis redis-cli TTL cuentas::002180000000000001
```

```
"{\"clabe\":\"0021800000000000001\",\"titular\":\"Ana\",\"saldo\":1000.00,\"
    leidaEn\":\"2026-11-07T03:59:10.516201245Z\"}"
(integer) 30
```

1.4 Invalidación tras una transferencia

Mandamos una transferencia de 200 pesos de Ana a Beto:

```
curl -X POST localhost:8080/api/transferencias -H 'Content-Type: application/json
    ' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","
      monto":200}'
```

La transferencia 64 quedó `COMPLETADA` con `saldoOrigenTrasCargo` de `600.00`. Enseguida preguntamos a Redis si la clave de Ana seguía existiendo:

```
docker compose exec redis redis-cli EXISTS cuentas::002180000000000001
# (integer) 0
```

La entrada había sido eliminada por el `@CacheEvict` en cuanto se confirmó el cargo. Al consultar de nuevo a Ana, el endpoint ya reportaba `saldo: 600.00`.

1.5 Medición del hit ratio

El *hit ratio* mide qué fracción de las lecturas fue resuelta por la caché:

$$\text{Hit Ratio} = \frac{\text{keyspace_hits}}{\text{keyspace_hits} + \text{keyspace_misses}}$$

Escenario A: 100 consultas seguidas sin escrituras.

```
docker compose exec redis redis-cli CONFIG RESETSTAT
for i in $(seq 1 100); do curl -s -o /dev/null localhost:8080/api/cuentas
    /002180000000000001; done
docker compose exec redis redis-cli INFO stats | grep keyspace
```

Obtuvimos `keyspace_hits:99` y `keyspace_misses:1` (el único *miss* fue la primera lectura, que calentó la caché):

$$\text{Hit Ratio}_A = \frac{99}{99+1} = 0.99 \quad (99.0\,\%)$$

Escenario B: 100 consultas con una transferencia cada 10 lecturas.

```
docker compose exec redis redis-cli CONFIG RESETSTAT
for i in $(seq 1 100); do
  if [ $((i % 10)) -eq 0 ]; then
    curl -s -X POST localhost:8080/api/transferencias -H 'Content-Type:
        application/json' \
      -d '{"claveOrigen":"002180000000000001","claveDestino
          ":"002180000000000002","monto":1}' > /dev/null
  fi
  curl -s -o /dev/null localhost:8080/api/cuentas/002180000000000001
```

```
done
docker compose exec redis redis-cli INFO stats | grep keyspace
```

Ahora el resultado fue `keyspace_hits:100` y `keyspace_misses:20`:

$$\text{Hit Ratio}_B = \frac{100}{100 + 20} \approx 0.8333 \quad (83.33\,\%)$$

La caída del hit ratio en el escenario B se explica porque cada transferencia dispara un `@CacheEvict` que elimina la entrada de Ana (y también la de Beto). La consulta que viene justo después ya no encuentra la clave, así que es un *miss* forzado que obliga a recargar desde PostgreSQL. En resumen: mientras más se modifica un dato, menos rinde una caché de lectura.

1.6 Medición de latencia: con caché y sin caché

Medimos 100 peticiones por corrida, calculando promedio, mediana y percentil 95, en tres corridas con caché y tres sin caché (`CACHE_TIPO=none`):

```
for i in $(seq 1 100); do
  curl -s -o /dev/null -w '%{time_total}\n' localhost:8080/api/cuentas
     /002180000000000001
done | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000} END {printf "promedio=%.2f ms
   mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
```

Para alternar entre modos usamos la variable `CACHE_TIPO` al recrear el servicio, y borramos la clave antes de la primera corrida con caché para partir de un *miss*:

```
# Con cache
docker compose exec redis redis-cli DEL cuentas::002180000000000001
# (3 corridas del comando anterior)

# Sin cache
CACHE_TIPO=none docker compose up -d cuenta
# (3 corridas del comando anterior)
```

Modo	Corrida	Prom. (ms)	Mediana (ms)	p95 (ms)	Lecturas a PG
Con caché	1	5.31	3.85	4.63	1
Con caché	2	5.79	4.16	7.34	0
Con caché	3	5.78	3.72	4.63	0
Sin caché	1	25.82	6.55	249.40	100
Sin caché	2	13.99	5.74	9.05	100
Sin caché	3	6.17	5.69	8.08	100

Cuadro 1: Latencia por corrida (PG = PostgreSQL). En las corridas 2 y 3 con caché la clave ya estaba en Redis.

Para confirmar el conteo de lecturas en las tres corridas sin caché ejecutamos `docker compose logs cuenta | grep -c "leida de Postgres"` y obtuvimos `300`: todas las consultas fueron a la base.

Sobre estos números hay que considerar que, en una máquina local, PostgreSQL ya tiene sus páginas en memoria y no hay red de por medio, así que la diferencia de velocidad es pequeña: la mediana

con caché se queda cerca de 3.8 ms y sin caché sube a unos 6.0 ms, con un pico de 249 ms en la primera corrida (arranque de conexiones). Lo realmente importante es otra cosa: con caché, de 300 consultas solo **1** llegó a PostgreSQL; sin caché llegaron las **300**.

Parte 2: Los avisos en MongoDB

2.1 Implementación en el servicio `notificacion`

La lógica quedó concentrada en la clase `BitacoraDeAvisos`:

- `documento(evento, destinatario, contacto)` construye un `org.bson.Document` con `_id` compuesto (`eventoId:clabe`) y los atributos comunes (`eventoId`, `transferenciaId`, `clabe`, `tipo`, `monto`, `saldo`, `fecha`), usando `Decimal128` para los importes y `Date` para los instantes. Con *pattern matching* sobre la interfaz sellada `Contacto` se agregan los campos propios de cada canal: `SMS` (`telefono`, `texto`), `CORREO` (`correo`, `asunto`, `cuerpo`) y `PUSH` (`token`, `titulo`, `cuerpo`).
- `registrar(evento)` recorre a los destinatarios (origen y destino) e inserta un documento por cada uno en la colección `avisos`. Si salta `DuplicateKeyException`, la ignora, porque significa que ese aviso ya se había guardado.
- `crearIndice()` crea el índice compuesto `clabe_fecha` con `{ clabe: 1, fecha: -1 }`.
- `ultimos(clabe)` devuelve los 10 avisos más recientes de una cuenta, ordenados por fecha descendente.

En `ConsumidorDeTransferencias` sustituimos la bandeja de PostgreSQL por `bitacora.registrar(evento)`. Con esto evitamos a propósito una doble escritura distribuida sin transacción.

2.2 Avisos por canal

La transferencia de Ana a Beto (SMS y correo) es la que ya habíamos hecho en la Parte 1 (transferencia 64). Para ver también el canal push agregamos una de 50 pesos de Ana a Carla (SMS y push) y luego consultamos la colección:

```
curl -s -X POST localhost:8080/api/transferencias -H 'Content-Type: application/
    json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000003","
      monto":50}'
docker compose exec mongo mongosh notificacion --eval 'db.avisos.find().sort({
    fecha: 1})'
```

Mostramos tres de los documentos obtenidos, uno por canal, para ver que cada uno trae atributos distintos en la misma colección:

```
{ _id: '3b090b47-...-3abe51967879:002180000000000001',
  tipo: 'TRANSFERENCIA_ENVIADA', monto: Decimal128('200'), saldo: Decimal128
      ('600.00'),
  canal: 'SMS', telefono: '+52 55 5555 0101',
  texto: 'TRANSFERENCIA_ENVIADA por 200, saldo resultante 600.00' }

{ _id: '3b090b47-...-3abe51967879:002180000000000002',
```

```
  tipo: 'TRANSFERENCIA_RECIBIDA', monto: Decimal128('200'), saldo: Decimal128
      ('700.00'),
  canal: 'CORREO', correo: 'beto@correo.mx', asunto: 'TRANSFERENCIA_RECIBIDA',
  cuerpo: 'TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.00' }

{ _id: '82ce7caa-...-63d5f9fd1de3:002180000000000003',
  tipo: 'TRANSFERENCIA_RECIBIDA', monto: Decimal128('50'), saldo: Decimal128
      ('350.00'),
  canal: 'PUSH', token: 'fcm:demo-002180000000000003', titulo: '
      TRANSFERENCIA_RECIBIDA',
  cuerpo: 'TRANSFERENCIA_RECIBIDA por 50, saldo resultante 350.00' }
```

2.3 Esquema en lectura

Para comprobar que no hay un esquema fijo, filtramos solo los avisos que tienen el campo `telefono`:

```
docker compose exec mongo mongosh notificacion --eval 'db.avisos.find({telefono:
    {$exists: true}}, {clabe: 1, canal: 1, telefono: 1})'
```

La consulta devolvió únicamente los avisos de canal SMS (los de Ana en las dos transferencias), sin necesidad de columnas vacías para los demás canales.

2.4 Endpoint de avisos a través del gateway

Consultamos los avisos de Beto después de 12 transferencias:

```
curl -s localhost:8080/api/notificaciones/002180000000000002 | jq .
curl -s localhost:8080/api/notificaciones/002180000000000002 | jq length
```

El endpoint regresó un arreglo con **exactamente 10 avisos** (`jq length` imprime `10`), ordenados del más reciente (saldo `710.00`) al más antiguo dentro del límite (saldo `701.00`), con lo que comprobamos el tope de 10 elementos.

2.5 Plan de ejecución con `explain`

Primero confirmamos que el índice existe y después comparamos la misma consulta de los últimos 10 avisos con el índice y forzando un escaneo completo.

```
docker compose exec mongo mongosh notificacion --eval 'db.avisos.getIndexes()'
```

```
# Con indice (clabe_fecha)
docker compose exec mongo mongosh notificacion --eval \
  'db.avisos.find({clabe: "002180000000000002"}).sort({fecha: -1}).limit(10).
      explain("executionStats")'
# Forzando escaneo completo
docker compose exec mongo mongosh notificacion --eval \
  'db.avisos.find({clabe: "002180000000000002"}).sort({fecha: -1}).limit(10).hint
      ({$natural: 1}).explain("executionStats")'
```

Métrica	Con índice	Sin índice
Etapas del plan ganador	LIMIT, FETCH, IXSCAN	SORT, COLLSCAN
`totalKeysExamined`	10	0
`totalDocsExamined`	10	24
`nReturned`	10	10
Ordenamiento en memoria	No	Sí (4,480 bytes)

Cuadro 2: Resultados de `explain(.executionStats")`.

Con el índice `clabe_fecha`, MongoDB salta directo a las 10 claves necesarias y no ordena nada, porque el índice ya guarda las claves en orden. Sin él, tuvo que leer los 24 documentos de la colección y ordenarlos en memoria. Con 24 documentos es irrelevante, pero con millones de avisos el escaneo completo saturaría la RAM y el disco.

Parte 3: Rompe algo: el evento duplicado

La mensajería de RabbitMQ entrega *al-menos-una-vez*. Si la red parpadea antes del ack, el broker vuelve a entregar el mensaje (`redelivered=true`). Para simular ese duplicado publicamos dos veces exactamente el mismo evento en el exchange `transferencias`, usando la API REST de RabbitMQ (puerto 15672):

```
curl -u mexibanco:mexibanco -X POST localhost:15672/api/exchanges/%2F/
    transferencias/publish \
  -H 'Content-Type: application/json' \
  -d '{"properties":{"delivery_mode":2,"content_type":"application/json"},"
      routing_key":"transferencia.realizada","payload_encoding":"string","payload
      ":"{\"eventoId\":\"11111111-2222-4333-8444-555555555555\",\"tipo\":\"
      TransferenciaRealizada\",\"transferenciaId\":9001,\"clabeOrigen
      \":\"002180000000000091\",\"clabeDestino\":\"002180000000000092\",\"monto
      \":25,\"saldoOrigen\":475,\"saldoDestino\":525,\"fecha\":\"2026-10-31T13
      :00:00Z\"}"}'
# (se ejecuto una segunda vez, identico)
```

Las dos peticiones respondieron `{routed":true}`. Después contamos cuántos documentos quedaron para ese `eventoId` y revisamos el log del consumidor:

```
docker compose exec mongo mongosh notificacion --eval \
  'db.avisos.countDocuments({eventoId: "11111111-2222-4333-8444-555555555555"})'
# 2
docker compose exec mongo mongosh notificacion --eval \
  'db.avisos.find({eventoId: "11111111-2222-4333-8444-555555555555"}, {clabe: 1,
      tipo: 1, canal: 1})'
docker compose logs notificacion | grep -E "9001"
```

```
[consumidor] TransferenciaRealizada de la transferencia 9001 (evento
    11111111-...-555555555555, redelivered=false): 2 aviso(s) nuevo(s)
[consumidor] TransferenciaRealizada de la transferencia 9001 (evento
    11111111-...-555555555555, redelivered=false): 0 aviso(s) nuevo(s)
```

Lo que pasó es que un evento genera dos avisos: uno para el origen y otro para el destino. La primera entrega los insertó (`2 aviso(s) nuevo(s)`). En la segunda, MongoDB rechazó ambas inserciones

por colisión en la clave `_id`; `BitacoraDeAvisos` atrapó la `DuplicateKeyException`, regresó `0` avisos nuevos y el consumidor confirmó el mensaje con `basicAck`. El resultado es el mismo que si el mensaje se hubiera procesado una sola vez, y sin necesidad de una tabla aparte como la de `evento_procesado` de la S09.

Parte 4: Rompe algo: el dato viejo

Una caché es una proyección derivada, no la fuente de la verdad, y puede quedarse con información obsoleta si alguien modifica la base por fuera de la aplicación. Lo comprobamos con este experimento:

1. Consultamos a Ana (`curl -s localhost:8080/api/cuentas/002180000000000001`) para poblar la caché: `saldo: 490.00`, `leidaEn: 2026-11-07T04:19:20Z`.
2. Modificamos directamente PostgreSQL, como si fuera un script de migración externo:

```
docker compose exec db psql -U mexibanco -d cuenta \
  -c "update cuenta set saldo = 5 where clabe = '002180000000000001'"
```

3. Monitoreamos el TTL y el saldo visible cada 5 segundos:

```
for i in $(seq 1 7); do
  docker compose exec redis redis-cli TTL cuentas::002180000000000001
  curl -s localhost:8080/api/cuentas/002180000000000001
  echo ""
  sleep 5
done
```

TTL (s)	Saldo que vio el cliente
26	490.00 (`leidaEn` 04:19:20)
16	490.00 (`leidaEn` 04:19:20)
10	490.00 (`leidaEn` 04:19:20)
5	490.00 (`leidaEn` 04:19:20)
−2 (clave expirada)	5.00 (*miss*, leído de PostgreSQL a las 04:20:47)

Cuadro 3: Evolución del saldo visible mientras la caché estaba desfasada.

¿Cuántos segundos vio el cliente un saldo que ya no era real? Durante **27 segundos**, el tiempo que le quedaba al TTL de 30 segundos. El campo `leidaEn` se mantuvo fijo en `04:19:20`, lo que le indica al cliente cuándo se leyó ese dato de la base.

4.1 Una transferencia mientras la caché mentía

Con Redis todavía afirmando que Ana tenía 490 pesos, intentamos transferir 100 a Beto:

```
curl -s -i -X POST localhost:8080/api/transferencias -H 'Content-Type:
    application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","
      monto":100}'
```

```
HTTP/1.1 422 Unprocessable Entity
{ "status": 422, "detail": "Saldo insuficiente en la CLABE 0021800000000000001", "
    servicio": "cuenta" }
```

¿Qué contestó la transferencia y por qué? Respondió `HTTP 422` con «Saldo insuficiente». El cargo no lee nunca de Redis: la decisión de mover dinero se toma siempre en PostgreSQL, dentro de una transacción local protegida por `@Version`. Como el saldo real era de 5 pesos, rechazó correctamente los 100 y no hubo sobregiro, aunque la caché dijera lo contrario.

Reto opcional: Redis se cae

El problema: una caché no debe tirar el sistema

Con la configuración estándar de Spring Cache, si Redis se apaga pasan dos cosas:

1. `GET /cuentas/{clabe}` lanza `RedisConnectionFailureException` y responde `HTTP 500`, a pesar de que PostgreSQL sigue vivo.
2. En una transferencia, el cargo sí se ejecuta en PostgreSQL, pero al hacer commit el `transactionAware()` intenta el `evict` en `afterCommit()`. Con Redis caído, esa excepción no se captura y Spring responde `HTTP 500`.

El segundo punto es el grave: el servicio `transferencia` recibe un 500 y cree que el cargo falló, pero el dinero **ya fue descontado**. Esto rompe el cuadre financiero por culpa de un componente que solo es una optimización.

La solución: un decorador resiliente

En `ConfiguracionCache.java` envolvimos la caché con `ResilientCache`, que atrapa los fallos de conexión, los registra con un `WARN` y sigue adelante. Una lectura fallida se trata como *miss* (se degrada a PostgreSQL) y un `put` o `evict` fallido simplemente se omite.

```
static class ResilientCache implements Cache {
    private final Cache delegate;
    ResilientCache(Cache delegate) { this.delegate = delegate; }

    @Override
    public ValueWrapper get(Object key) {
        try {
            return delegate.get(key);
        } catch (RuntimeException e) {
            log.warn("[cache] fallo al leer clave {} de Redis: {}", key, e.
                getMessage());
            return null; // se trata como cache miss y se degrada a Postgres
        }
    }

    @Override
    public void evict(Object key) {
        try {
            delegate.evict(key);
        } catch (RuntimeException e) {
```

```
            log.warn("[cache] fallo al borrar (DEL) clave {} de Redis: {}", key,
                e.getMessage());
        }
    }
    // put(...) sigue el mismo patron
}
```

El `ResilientCacheManager` devuelve cada caché como `new TransactionAwareCacheDecorator(new ResilientCache(cache))`, de modo que se conserva la regla de invalidar solo después del commit.

Prueba con Redis apagado

Detuvimos el contenedor de Redis y probamos tres cosas:

```
docker compose stop redis

# 1. Consulta de saldo
curl -s -i localhost:8080/api/cuentas/002180000000000001

# 2. Transferencia de 10 pesos
curl -s -i -X POST localhost:8080/api/transferencias -H 'Content-Type:
    application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","
      monto":10}'

# 3. Logs del servicio cuenta
docker compose logs cuenta | grep -C 1 "fallo al"
```

1. **Consulta de saldo.** `GET /api/cuentas/002180000000000001` respondió `HTTP 200` con `saldo: 460.00`: al fallar Redis se leyó de PostgreSQL de forma transparente.
2. **Transferencia de 10 pesos.** Respondió `HTTP 201 Created`, estado `COMPLETADA`, con `eventoId 646cb4eb-a262-4259-ba08-7467db27a79d`. La saga terminó bien aunque Redis no estuviera.
3. **Logs de cuenta.** Aparecieron las advertencias de los fallos de Redis, pero ninguna excepción llegó al cliente:

```
[cuenta] WARN ConfiguracionCache : [cache] fallo al leer clave 002180000000000001
    de Redis: Unable to connect to Redis
[cuenta] INFO ConsultaDeSaldo    : [consulta] 002180000000000001 leida de
    Postgres
[cuenta] WARN ConfiguracionCache : [cache] fallo al borrar (DEL) clave
    002180000000000001 de Redis: Unable to connect to Redis
[cuenta] WARN ConfiguracionCache : [cache] fallo al borrar (DEL) clave
    002180000000000002 de Redis: Redis command timed out
```

Finalmente encendimos Redis y, tras esperar un par de segundos, volvimos a consultar:

```
docker compose start redis
sleep 2 && curl -s -i localhost:8080/api/cuentas/002180000000000001
docker compose exec redis redis-cli PING
```

La consulta respondió `HTTP 200` con `saldo: 450.00`, y la caché volvió a funcionar sin reiniciar ningún microservicio.

Preguntas de cierre

1. ¿Por qué MongoDB para los avisos y no para el saldo? Usa lo que viste: la forma de cada aviso, cómo se leen y qué tendría que garantizar el saldo.
Los avisos cambian de forma según el canal: un SMS lleva `telefono` y `texto`, un correo lleva `correo`, `asunto` y `cuerpo`, y un push lleva `token`, `titulo` y `cuerpo`. En MongoDB todos conviven en una sola colección; en PostgreSQL tendríamos columnas casi siempre vacías o varias tablas con llaves foráneas. Además, un aviso se escribe una vez y no se modifica, y se lee completo por cuenta y ordenado por fecha, sin joins. El saldo es lo contrario: es la verdad del dinero y necesita garantías ACID, `@Version` y transacciones atómicas para evitar dobles gastos, así que se queda en PostgreSQL.

2. ¿Qué hit ratio midieron en A y en B, y por qué cambia? Con tus números de latencia y de lecturas a Postgres: ¿qué compraste con la caché, velocidad o capacidad?
En A el hit ratio fue de 99.0 % y en B de 83.33 %. Baja porque cada transferencia invalida con `DEL` las claves de las cuentas involucradas, y la siguiente lectura es un miss obligado. Lo que compramos fue sobre todo **capacidad**: la mediana apenas pasó de unos 6.0 ms a unos 3.8 ms, pero 100 consultas pasaron de 100 lecturas a PostgreSQL a una sola, más de 99 % menos carga para la base.

3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché? Usa lo que viste en la Parte 4.
Habría sobregiros. En la Parte 4 PostgreSQL tenía $5.00 mientras Redis siguió mostrando $490.00 durante 27 segundos. Si el cargo se autorizara con la caché, habría aprobado transferencias de hasta $490.00 sobre una cuenta con $5.00 reales, dejando saldo negativo y un descuadre contra `movimiento`. Por eso la consulta que autoriza no usa caché y Redis solo sirve para mostrar el saldo al cliente.

4. Al aplicar un movimiento, ¿por qué se hace DEL de la copia y no SET con el saldo nuevo? ¿Y por qué después del commit y no antes?
Se usa `DEL` porque con dos transacciones concurrentes los `SET` pueden llegar a Redis en desorden y dejar 800 aunque el saldo real sea 600. El `DEL` no tiene ese riesgo: obliga a que la siguiente lectura traiga el valor definitivo de PostgreSQL, y de paso no escribimos un dato que quizá nadie vuelva a consultar.

Va después del commit porque, si se borra antes y la transacción falla, la invalidación fue en vano. Peor aún, una lectura en ese instante leería el saldo viejo de PostgreSQL y lo volvería a guardar en Redis hasta que expire el TTL. Con `transactionAware()`, el `evict` corre en `afterCommit()`, cuando el saldo nuevo ya está fijo.

Conclusión

Este laboratorio nos mostró que la persistencia políglota no consiste en usar más tecnologías por moda, sino en decidir con criterio qué dato es la verdad y cuál es una copia. PostgreSQL conserva la autoridad sobre el dinero; Redis acelera y protege las lecturas, pero su dato puede estar viejo hasta 30 segundos, por lo que nunca participa en decisiones de dinero; y MongoDB almacena avisos heterogéneos con idempotencia natural gracias a su clave compuesta.

También vimos que cada pieza nueva trae sus propios riesgos: la caché puede mentir, el broker puede entregar duplicados y una caché caída no debe tumbar una transferencia. Los tres experimentos (el

duplicado, el dato viejo y el reto de Redis apagado) confirmaron que, si el diseño separa bien la verdad de lo derivado, esas fallas se quedan contenidas y no llegan al cliente ni al dinero.